



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**INFORME DE LABORATORIO: POO
MATRICES, ÁREAS Y PERÍMETROS**

Curso: PROGRAMACIÓN II – Sección: B

Docente: Ing. Breno Luque Zuñiga

ALUMNO: Yordan Antonio Cohaila Mamani

Contenido

I. INFORMACIÓN GENERAL	3
Objetivos	3
Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEÓRICO	3
Programación Orientada a Objetos	3
Clase.....	4
Objeto	4
Encapsulamiento	4
Herencia	4
Polimorfismo.....	4
Matrices.....	5
Área y perímetro	5
III. PROCEDIMIENTO	5
Paso 1: Creación del proyecto en Visual Studio	5
Paso 2: Desarrollo de la interfaz para multiplicación de matrices	5
Paso 3: Creación de la clase Matriz	6
Paso 4: Programación de la multiplicación de matrices.....	7
Paso 5: Desarrollo de la interfaz para área y perímetro de figuras	7
Paso 6: Creación de la clase base Figura	8
Paso 7: Creación de las clases derivadas	8
Paso 8: Programación del cálculo de área y perímetro	8
IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS	9
V. CUESTIONARIO	10
CONCLUSIONES	11
RECOMENDACIONES.....	11

I. INFORMACIÓN GENERAL

Objetivos

- Aplicar los conceptos básicos de Programación Orientada a Objetos mediante el desarrollo de aplicaciones en C#.
- Crear una aplicación que permita realizar la multiplicación de dos matrices.
- Diseñar una clase llamada Matriz que permita almacenar, ingresar, mostrar y multiplicar matrices.
- Implementar encapsulamiento mediante atributos privados y métodos públicos.
- Crear una aplicación para calcular el área y perímetro de diferentes figuras geométricas.
- Aplicar herencia y polimorfismo mediante una clase base Figura y clases derivadas.
- Diseñar interfaces gráficas en Windows Forms que permitan ingresar datos, procesarlos y mostrar resultados.

Equipos, materiales, programas y recursos utilizados

- Computadora o laptop con sistema operativo Windows.
- Microsoft Visual Studio.
- Lenguaje de programación C#.
- Windows Forms.
- .NET Framework.
- Controles gráficos como Label, TextBox, Button, ComboBox y Panel.
- Documento guía de la práctica.
- Conocimientos básicos de matrices, áreas, perímetros y Programación Orientada a Objetos.

II. MARCO TEÓRICO

Programación Orientada a Objetos

La Programación Orientada a Objetos, también conocida como POO, es un paradigma de programación que permite organizar el código mediante clases y objetos. Este enfoque facilita que el programa sea más ordenado, reutilizable y fácil de mantener.

En la práctica se aplicó POO en dos ejercicios. En el primer ejercicio se creó una clase Matriz para realizar operaciones con matrices. En el segundo ejercicio se creó una clase base Figura y varias clases derivadas para calcular área y perímetro.

Clase

Una clase es una estructura que funciona como molde para crear objetos. En ella se definen atributos y métodos.

En el primer ejercicio se creó la clase Matriz, que contiene las filas, columnas y datos de una matriz. En el segundo ejercicio se creó la clase Figura, de la cual heredan las clases Cuadrado, Rectangulo, Triangulo y Circulo.

Objeto

Un objeto es una instancia de una clase. Por ejemplo, en el programa de matrices se crean objetos como:

```
Matriz matrizA = new Matriz(filasA, columnasA);
```

```
Matriz matrizB = new Matriz(filasB, columnasB);
```

Estos objetos representan las matrices que el usuario ingresa en la interfaz.

Encapsulamiento

El encapsulamiento consiste en proteger los datos internos de una clase usando atributos privados y métodos públicos. Esto permite controlar mejor el acceso a la información.

En la clase Matriz, los datos se almacenan en atributos privados y se accede a ellos mediante métodos como IngresarValor() y MostrarValor().

Herencia

La herencia permite que una clase derive de otra y reutilice sus características. En el segundo ejercicio, las clases Cuadrado, Rectangulo, Triangulo y Circulo heredan de la clase base Figura.

Ejemplo:

```
public class Cuadrado : Figura
```

Esto significa que Cuadrado es una figura y debe implementar los métodos para calcular área y perímetro.

Polimorfismo

El polimorfismo permite usar una misma variable de tipo general para almacenar objetos de diferentes clases derivadas. En el segundo ejercicio se utilizó una variable de tipo Figura para trabajar con cualquier figura seleccionada.

Ejemplo:

```
Figura figura = null;
```

```
figura = new Cuadrado(lado);
```

```
figura = new Circulo(radio);
```

De esta forma, el programa puede llamar a los métodos `CalcularArea()` y `CalcularPerimetro()` sin importar qué figura se haya seleccionado.

Matrices

Una matriz es una estructura formada por filas y columnas. Cada posición almacena un valor numérico. Para multiplicar dos matrices, se debe cumplir la siguiente condición:

Columnas de la matriz A = Filas de la matriz B

Si se cumple esta condición, el resultado tendrá la cantidad de filas de la matriz A y la cantidad de columnas de la matriz B.

Área y perímetro

El área representa la superficie que ocupa una figura, mientras que el perímetro representa la suma de sus lados o el contorno total de la figura.

En la práctica se trabajó con las siguientes figuras:

- Cuadrado.
- Rectángulo.
- Triángulo.
- Círculo.

III. PROCEDIMIENTO

Paso 1: Creación del proyecto en Visual Studio

Se creó un proyecto en Microsoft Visual Studio utilizando el lenguaje C# y el tipo de aplicación Windows Forms. Este tipo de proyecto permitió diseñar interfaces gráficas mediante formularios y controles visuales.

El proyecto se desarrolló aplicando Programación Orientada a Objetos, separando la lógica de cálculo en clases y utilizando el formulario solamente para ingresar datos, ejecutar botones y mostrar resultados.

Paso 2: Desarrollo de la interfaz para multiplicación de matrices

Se diseñó una interfaz gráfica para realizar la multiplicación de dos matrices. En la parte superior del formulario se colocó el título “**Multiplicación de Matrices**”.

Luego se agregaron campos de texto para ingresar las filas y columnas de la matriz A y de la matriz B. Para la matriz A se usaron los campos `txtFilasA` y `txtColumnasA`. Para la matriz B se usaron los campos `txtFilasB` y `txtColumnasB`.

También se agregaron tres botones principales. El botón `btnCrearMatriz` permite generar automáticamente los espacios donde el usuario escribe los valores de cada matriz. El botón `btnMultiplicar` ejecuta la multiplicación de matrices. El botón `btnBorrar` limpia todos los datos ingresados y el resultado.

Además, se colocaron tres paneles. El panel pnlMatrizA muestra los campos de la matriz A, el panel pnlMatrizB muestra los campos de la matriz B y el panel pnlMatrizResultado muestra el resultado de la multiplicación.

Esta interfaz permite que el usuario ingrese las dimensiones de las matrices, cree los campos de entrada, escriba los valores, realice la operación y visualice la matriz resultante.

Paso 3: Creación de la clase Matriz

Se creó una clase llamada Matriz, encargada de representar una matriz dentro del programa. Esta clase contiene atributos privados para almacenar el número de filas, el número de columnas y los datos de la matriz.

También se implementaron métodos públicos para ingresar valores, mostrar valores y multiplicar matrices.

El uso de esta clase permitió aplicar encapsulamiento, ya que los datos internos de la matriz no se manipulan directamente desde el formulario, sino mediante métodos.

La estructura principal de la clase fue la siguiente:

```
1  using System;
2
3  namespace P00_Matrices
4  {
5      13 referencias
6      public class Matriz
7      {
8          private int filas;
9          private int columnas;
10         private double[,] datos;
11
12         5 referencias
13         public int Filas
14         {
15             get { return filas; }
16         }
17
18         6 referencias
19         public int Columnas
20         {
21             get { return columnas; }
22         }
23
24         0 referencias
25         public double[,] Datos
26         {
27             get { return datos; }
28         }
29
30         3 referencias
31         public Matriz(int filas, int columnas)
32         {
33             this.filas = filas;
34             this.columnas = columnas;
35             datos = new double[filas, columnas];
36         }
37
38         3 referencias
39         public void IngresarValor(int fila, int columna, double valor)
40         {
41             datos[fila, columna] = valor;
42         }
43
44         3 referencias
45         public double MostrarValor(int fila, int columna)
46         {
47             return datos[fila, columna];
48         }
49     }
50 }
```

```

43 1 referencia
44 public static Matriz Multiplicar(Matriz matrizA, Matriz matrizB)
45 {
46     if (matrizA.Columnas != matrizB.Filas)
47     {
48         throw new Exception("No se pueden multiplicar las matrices.");
49     }
50     Matriz resultado = new Matriz(matrizA.Filas, matrizB.Columnas);
51     for (int i = 0; i < matrizA.Filas; i++)
52     {
53         for (int j = 0; j < matrizB.Columnas; j++)
54         {
55             double suma = 0;
56             for (int k = 0; k < matrizA.Columnas; k++)
57             {
58                 suma += matrizA.MostrarValor(i, k) * matrizB.MostrarValor(k, j);
59             }
60             resultado.IngresarValor(i, j, suma);
61         }
62     }
63     return resultado;
64 }
65 }
66 }
67 }
68 }
69 }
70 }

```

Paso 4: Programación de la multiplicación de matrices

Se programó el botón Crear Matriz para leer las dimensiones ingresadas por el usuario. Luego se validó que las filas y columnas sean mayores que cero.

También se verificó que las columnas de la matriz A sean iguales a las filas de la matriz B. Esta validación es necesaria porque, si no se cumple, la multiplicación no puede realizarse.

Después de validar los datos, el programa genera dinámicamente los TextBox dentro de los paneles pnlMatrizA y pnlMatrizB.

Luego se programó el botón Multiplicar. Este botón lee los valores ingresados, crea los objetos matrizA y matrizB, llama al método Multiplicar() de la clase Matriz y finalmente muestra la matriz resultante dentro del panel pnlMatrizResultado.

El botón Borrar fue programado para limpiar los campos de filas, columnas, matrices y resultado.

Paso 5: Desarrollo de la interfaz para área y perímetro de figuras

Se diseñó una segunda interfaz gráfica para calcular el área y perímetro de figuras geométricas. En la parte superior del formulario se colocó el título “Área y Perímetro de Figuras”.

Se agregó un ComboBox llamado cmbFigura, el cual permite seleccionar una figura geométrica entre las opciones: cuadrado, rectángulo, triángulo y círculo.

También se agregaron tres campos de entrada llamados txtMedida1, txtMedida2 y txtMedida3. Estos campos cambian su significado según la figura seleccionada. Por ejemplo, para el cuadrado solo se utiliza una medida, que representa el lado; para el rectángulo se utilizan base y altura; para el triángulo se utilizan tres lados; y para el círculo se utiliza el radio.

Además, se agregaron dos campos de salida llamados txtArea y txtPerimetro, los cuales muestran los resultados obtenidos. Estos campos se configuraron como solo lectura para evitar que el usuario modifique directamente los resultados.

Finalmente, se agregaron dos botones: btnCalcular, encargado de realizar los cálculos, y btnLimpiar, encargado de borrar los datos ingresados.

Paso 6: Creación de la clase base Figura

Se creó una clase base abstracta llamada Figura. Esta clase define los métodos generales que todas las figuras deben tener: calcular área y calcular perímetro.

La clase fue declarada como abstracta porque no se desea crear objetos directamente de Figura, sino usarla como base para otras clases.

```
public abstract class Figura
{
    public abstract double CalcularArea();

    public abstract double CalcularPerimetro();
}
```

Paso 7: Creación de las clases derivadas

Se crearon cuatro clases derivadas: Cuadrado, Rectangulo, Triangulo y Circulo.

Cada clase hereda de Figura y sobrescribe los métodos CalcularArea() y CalcularPerimetro() según las fórmulas correspondientes.

La clase Cuadrado calcula el área multiplicando lado por lado y el perímetro multiplicando el lado por cuatro.

La clase Rectangulo calcula el área multiplicando base por altura y el perímetro mediante la fórmula $2 * (base + altura)$.

La clase Triangulo calcula el perímetro sumando sus tres lados. Para el área se utilizó la fórmula de Herón, ya que la interfaz permite ingresar tres lados. Además, se agregó un método llamado EsValido() para comprobar que los lados ingresados formen un triángulo válido.

La clase Circulo calcula el área usando $\pi * radio^2$ y el perímetro usando $2 * \pi * radio$.

Paso 8: Programación del cálculo de área y perímetro

En el formulario se programó el evento Form1_Load para cargar las figuras dentro del ComboBox.

También se programó el evento SelectedIndexChanged del ComboBox, con el objetivo de cambiar los textos de las medidas según la figura seleccionada. Por ejemplo, si se

selecciona círculo, solo se muestra el campo del radio. Si se selecciona triángulo, se muestran tres campos para ingresar los lados.

En el botón **Calcular** se creó una variable de tipo Figura. Según la opción elegida en el ComboBox, esta variable puede recibir un objeto de tipo Cuadrado, Rectangulo, Triangulo o Circulo.

Ejemplo:

```
Figura figura = null;
```

```
figura = new Cuadrado(lado);
```

```
figura = new Rectangulo(baseRectangulo, altura);
```

```
figura = new Circulo(radio);
```

Esto permitió aplicar polimorfismo, ya que una variable de tipo Figura puede trabajar con diferentes clases derivadas.

Finalmente, se llamaron los métodos:

```
figura.CalcularArea();
```

```
figura.CalcularPerimetro();
```

Los resultados se mostraron en los campos txtArea y txtPerimetro.

IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

¿Qué indican los resultados?

Los resultados indican que se logró desarrollar correctamente dos aplicaciones en C# Windows Forms aplicando Programación Orientada a Objetos.

En el primer ejercicio, la aplicación permitió ingresar las dimensiones de dos matrices, generar automáticamente los campos para sus valores, validar si la multiplicación era posible y mostrar la matriz resultante.

En el segundo ejercicio, la aplicación permitió seleccionar una figura geométrica, ingresar sus medidas y calcular correctamente su área y perímetro.

También se comprobó el uso de clases, objetos, métodos, encapsulamiento, herencia y polimorfismo. Estos conceptos permitieron organizar el código y separar la lógica de cálculo de la interfaz gráfica.

¿Qué se ha encontrado?

Se encontró que el uso de clases permite ordenar mejor el código y evitar que toda la lógica esté escrita directamente en el formulario.

En el ejercicio de matrices, se comprobó que antes de multiplicar es obligatorio validar que las columnas de la matriz A sean iguales a las filas de la matriz B.

En el ejercicio de figuras geométricas, se comprobó que una clase base puede servir como modelo para varias clases derivadas. Esto permitió que cada figura tenga su propio cálculo de área y perímetro.

También se encontró que el polimorfismo permite trabajar con diferentes figuras usando una sola variable de tipo Figura, haciendo que el programa sea más flexible.

V. CUESTIONARIO

1. ¿Qué es la Programación Orientada a Objetos?

La Programación Orientada a Objetos es un paradigma de programación que organiza el código mediante clases y objetos. Permite aplicar conceptos como encapsulamiento, herencia y polimorfismo.

2. ¿Qué es una clase?

Una clase es una estructura que define atributos y métodos. Funciona como un molde para crear objetos.

3. ¿Qué es un objeto?

Un objeto es una instancia de una clase. Representa un elemento concreto dentro del programa.

4. ¿Qué es encapsulamiento?

El encapsulamiento consiste en proteger los datos internos de una clase usando atributos privados y permitiendo el acceso mediante métodos públicos.

5. ¿Cuál es la condición para multiplicar dos matrices?

La condición es que el número de columnas de la primera matriz sea igual al número de filas de la segunda matriz.

Columnas de A = Filas de B

6. ¿Qué es herencia?

La herencia permite que una clase derive de otra. En la práctica, las clases Cuadrado, Rectangulo, Triangulo y Circulo heredan de la clase base Figura.

7. ¿Qué es polimorfismo?

El polimorfismo permite que una variable de una clase base pueda almacenar objetos de distintas clases derivadas. En la práctica se usó una variable de tipo Figura para trabajar con diferentes figuras.

8. ¿Por qué se usó una clase abstracta Figura?

Se usó una clase abstracta porque Figura representa una idea general. No se necesita crear una figura genérica, sino figuras específicas como cuadrado, rectángulo, triángulo y círculo.

CONCLUSIONES

- Se logró desarrollar una aplicación para multiplicar matrices utilizando C# Windows Forms.
- Se creó una clase Matriz para almacenar datos, ingresar valores, mostrar valores y multiplicar matrices.
- Se aplicó encapsulamiento mediante atributos privados y métodos públicos.
- Se validó correctamente la condición necesaria para multiplicar matrices.
- Se desarrolló una segunda aplicación para calcular área y perímetro de figuras geométricas.
- Se aplicó herencia mediante una clase base Figura y clases derivadas.
- Se aplicó polimorfismo al usar una variable de tipo Figura para trabajar con diferentes figuras.
- Se comprobó que la Programación Orientada a Objetos permite organizar mejor el código y separar la lógica de cálculo de la interfaz gráfica.

RECOMENDACIONES

- Verificar que los nombres de los controles coincidan con los nombres usados en el código.
- Validar siempre que los datos ingresados sean numéricos y mayores que cero.
- En la multiplicación de matrices, comprobar que las dimensiones sean compatibles antes de realizar la operación.
- Mantener las clases en el mismo namespace para evitar errores al compilar.
- Separar la lógica de cálculo en clases y no colocar todo el código dentro del formulario.
- Probar el programa con diferentes valores para comprobar que los resultados sean correctos.
- Usar nombres claros para clases, métodos y controles, ya que esto facilita la corrección de errores.